

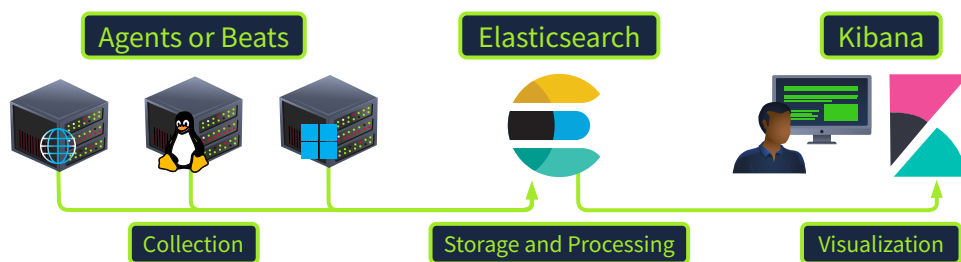
L2 - 3.2. Elastic: Setting up a SOC Lab

Task 1 - Introduction

The Elastic Stack enables security teams to ingest massive volumes of data and transform them into actionable insights. In this room, you'll explore Elastic's core architecture and get hands-on experience building a SOC lab to ingest, search, and investigate log data from multiple sources.

Task 2 - Elastic Stack Architecture

The Elastic Stack is a search and analysis platform designed to help analysts manage data from a wide variety of sources, forming the foundation of the modern Security Information and Event Management (SIEM) solution.



Stack Components

Elasticsearch: Core search engine and data store that indexes logs, alerts, and security data for fast searching and analysis; uses **TCP 9200** by default.

Kibana: Web-based interface for visualizing, searching, dashboarding, and investigating Elasticsearch data; uses **port 5601** and does not store data.

Elastic Agents: Lightweight endpoint services that collect and send logs, metrics, and security data to Elastic; replace individual Beats and make outbound connections.

Fleet Server: Centralized management service for Elastic Agents, handling deployment, policies, configuration, and updates; commonly uses **TCP 8220**.

Extended Elastic Components

- [Integrations\(opens in new tab\)](#): Elastic has created many integrations for third-party services, software, and infrastructure that allow for easy ingestion of logs and metrics
- [Beats\(opens in new tab\)](#): Lightweight data shippers designed to collect specific types of log data. Examples include Packetbeat for network data and Metricbeat for system metrics
- [Logstash\(opens in new tab\)](#): Used to ingest, parse, and transform data before it is indexed. Often used when logs require complex parsing or enrichment

In a typical Elastic-based deployment, data is collected from endpoints and servers by Elastic Agents and managed centrally using Fleet Server. The data is then ingested and indexed into Elasticsearch, making it searchable and available for analysis. Analysts interact with this data through Kibana, using dashboards, searches, and visualizations to conduct investigations and gain visibility across the environment.

Answer the questions below

Which component of the Elastic Stack represents the interface analysts use to visualize log data?

Kibana

Which Elastic component would you use to manage the agents deployed across your client's infrastructure?

Fleet Server

Task 3 - Deploying Elasticsearch and Kibana

In this task, we will install Elasticsearch and Kibana on a Linux machine.

The `.deb` installers have been sourced directly from the official Elastic website and have been placed in the `/home/ubuntu/Downloads/elastic` directory. In a production environment, you would typically download these using `wget` or `curl`, or by adding the Elastic APT repository to ensure you are installing the latest releases.

Installing Elasticsearch: Let's begin by switching over to the `root` user, navigating to the `Downloads/elastic` directory, and using the `dpkg` command to install

Elasticsearch. During installation, Elastic will create a password for the built-in superuser. Go ahead and mark this down, as you will need it later.

Elasticsearch Installation

```
ubuntu@soclabelastic:~$ sudo su
root@soclabelastic:/home/ubuntu# cd Downloads/elastic
root@soclabelastic:/home/ubuntu/Downloads/elastic# ls
elasticagent elasticsearch.deb kibana.debroot@soclabelastic:/
home/ubuntu/Downloads/elastic# dpkg -i elasticsearch.deb
Preparing to unpack elasticsearch.deb ...
Creating elasticsearch group... OK
Creating elasticsearch user... OK
Unpacking elasticsearch (9.2.4)
----- Security autoconfiguration inform
ation -----

Authentication and authorization are enabled.
TLS for the transport and HTTP layers is enabled and configur
ed.

The generated password for the elastic built-in superuser is
: <sample_password> # <- password
```

Setting Elasticsearch Resources: Using your favorite text editor, create the following file and add the two lines below. Doing so will ensure Elasticsearch runs efficiently while staying within the memory limits of your VM:

- `/etc/elasticsearch/jvm.options.d/heap.options`

```
-Xms1g
-Xmx1g
```

Starting Elasticsearch: Now that we have Elasticsearch installed and the resources set up, let's start it, enable it, and check its status. Enabling the service in your home lab ensures that Elasticsearch starts every time you boot your lab machine.

Elasticsearch Start

```
root@soclabelastic:/home/ubuntu/Downloads/elastic# systemctl
start elasticsearch
root@soclabelastic:/home/ubuntu/Downloads/elastic# systemctl
enable elasticsearch
root@soclabelastic:/home/ubuntu/Downloads/elastic# systemctl
status elasticsearch
● elasticsearch.service - Elasticsearch
   Loaded: loaded (/usr/lib/systemd/system/elasticsearch.ser
  vice; enabled; preset: enabled)
   Active: active (running) since Tue 2026-01-27 06:38:44 UT
 C; 42s ago
   ...
```

Installing Kibana: Next, we'll install Kibana, the web interface for the Elastic Stack. Ensure you are running as the `root` user, then use `dpkg` to install the Kibana package.

Kibana Installation

```
root@soclabelastic:/home/ubuntu/Downloads/elastic# dpkg -i ki
bana.deb
Preparing to unpack kibana.deb ...
Unpacking kibana (9.2.4) ...
Setting up kibana (9.2.4) ...
Creating kibana group... OK
Creating kibana user... OK
Created Kibana keystore in /etc/kibana/kibana.keystore
...
```

Kibana Configuration: Before starting Kibana, we will make two small additions to the configuration file to assist us with the Fleet server installation in the next task. Go ahead and open the following file in your text editor of choice, then add the two lines at the bottom of the file:

- `/etc/kibana/kibana.yml`

```
xpack.encryptedSavedObjects.encryptionKey: "soc-lab-training-  
key-32chars-long!"  
xpack.fleet.registryUrl: "http://localhost:8081"
```

Kibana Start

```
root@soclabelastic:/home/ubuntu/Downloads/elastic# systemctl  
start kibana  
root@soclabelastic:/home/ubuntu/Downloads/elastic# systemctl  
enable kibana  
root@soclabelastic:/home/ubuntu/Downloads/elastic# systemctl  
status kibana  
● kibana.service - Kibana  
   Loaded: loaded (/usr/lib/systemd/system/kibana.service; e  
nabled; preset: enabled)  
   Active: active (running) since Tue 2026-01-27 07:21:34 UT  
C; 5min ago  
       . . .
```

Accessing the UI: By default, Kibana binds to the `localhost` on port `5601`. Go ahead and open the browser in your lab machine and navigate to `http://localhost:5601/`.

Create an enrollment token `/usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token -s kibana`

1. Create a verification code `/usr/share/kibana/bin/kibana-verification-code`

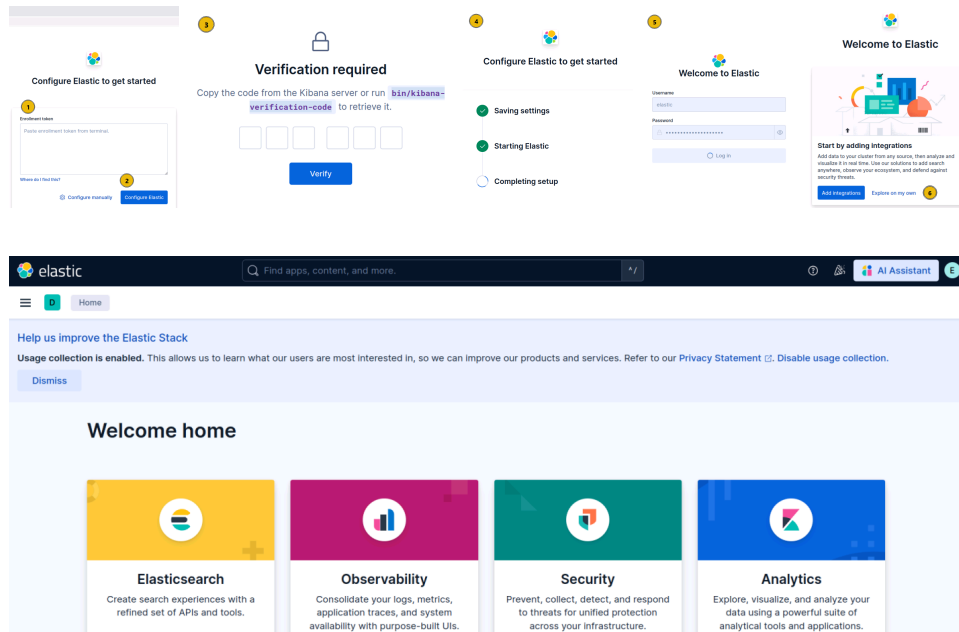
Once your enrollment token and verification code have been generated, head back to the browser:

1. Enter your enrollment token
2. Click `Configure Elastic` to continue
3. Enter your verification code and click `Verify`
4. Wait for Elastic and Kibana to set up
5. Enter the `elastic` username and the password generated during install

6. Click `Explore on my own`

If you missed the password during the Elasticsearch installation or need to reset it, you can use the command below to generate a new one:

- `/usr/share/elasticsearch/bin/elasticsearch-reset-password -u elastic`



Answer the questions below

Which port does the Kibana web interface run on by default?

5601

Access the Kibana menu in the top left of the Kibana home page. What is the name of the first section that is listed?

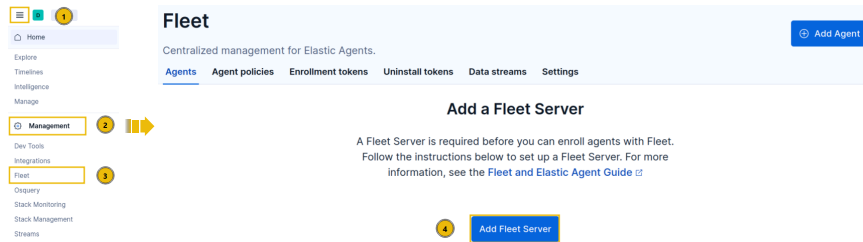
Analytics - in Kibana allows you to visualize, explore, and uncover actionable insights from your Elastic data using dashboards, maps, and reporting tools

Task 4 - Deploying Fleet Server and Elastic Agent

Fleet Installation: From the previous task, you should be on the Kibana home page. From here, we will begin setting up the Fleet server:

1. Click the Kibana menu in the top left

2. Scroll down to **Management**
3. Select **Fleet**
4. From the Fleet home page, click **Add Fleet Server**



In the Add a Fleet Server flyout, let's fill in the details of our server:

1. Enter the name **fleet-server**
2. Enter your lab machine IP as the URL **https://MACHINE_IP:8220**
3. Click **Generate Fleet Server policy**

From here, Elastic will generate a command for you to run in your terminal to install Fleet Server on your host. In your home lab or a production environment,

this command will download the Elastic Agent archive from Elastic, unzip it, and install Fleet Server. In this room, the executable has already been placed in the lab machine for ease of use. In the `Install Fleet Server to a centralized host` section:

1. Click the OS dropdown and select `Linux x86_64`
2. Copy the command starting at `./elastic-agent`

Add a Fleet Server

A Fleet Server is required before you can enroll agents with Fleet. Follow the instructions below to set up a Fleet Server. For more information, see the [Fleet and Elastic Agent Guide](#)

Quick Start Advanced

Install Fleet Server to a centralized host

Install Fleet Server agent on a centralized host so that other hosts you wish to monitor can connect to it. In production, we recommend using one or more dedicated hosts. For additional guidance, see our [Installation docs](#)

Linux aarch64	MacOS aarch64	DEB aarch64	RPM aarch64	...
---------------	---------------	-------------	-------------	-----

1

Windows x86_64
Windows MSI
Linux x86_64
MacOS x86_64
DEB x86_64
RPM x86_64

```
curl -L -O https://artifacts.elastic.co/downloads/
tar xzvf elastic-agent-9.2.4-linux-x86_64.tar.gz
cd elastic-agent-9.2.4-linux-x86_64
sudo ./elastic-agent install \
  --fleet-server-es=https://10.80.149.178:9200 \
  --fleet-server-service-token=AAEAAWVsYXN0aWMvZmx
  --fleet-server-policy=fleet-server-policy \
  --fleet-server-es-ca-trusted-fingerprint=fa0a67d
  --fleet-server-port=8220 \
  --install-servers
```

2

We will need to make one modification to the command you copied. Go ahead and add the line `--insecure` at the bottom of your command. The `--insecure` flag is required because this lab uses self-signed TLS certificates, allowing Elastic Agent to connect without certificate verification. Next, navigate to `/home/ubuntu/Downloads/elastic/elasticagent`, ensure you're running as `root`, and enter the command.

Note: You can use the `Pluma` text editor in your VM to edit the command.

Fleet Installation

```
root@soclabelastic:/home/ubuntu/Downloads/elastic# cd elastic
agent
root@soclabelastic:/home/ubuntu/Downloads/elastic/elasticagen
t# ./elastic-agent install \
```



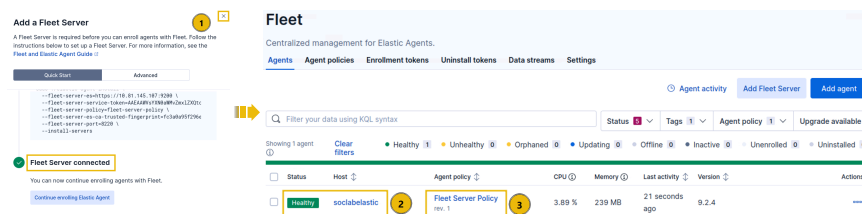
```
--fleet-server-es=https://MACHINE_IP:9200 \
--fleet-server-service-token=<your_token> \
--fleet-server-policy=fleet-server-policy \
--fleet-server-es-ca-trusted-fingerprint=<your_fingerprint> \
--fleet-server-port=8220 \
--install-servers \
--insecure
```

Elastic Agent will be installed at /opt/Elastic/Agent and will run as a service. Do you want to continue? [Y/n]:Y
 [==] Service Started [1m3s] Elastic Agent successfully installed, starting enrollment.

...
 Successfully enrolled the Elastic Agent.
 [==] Done [1m32s]
 Elastic Agent has been successfully installed.

Verifying Installation: You will see in your terminal output that the Elastic Agent has been successfully installed. Let's head back to Kibana to verify. If you look below the command that you previously copied, you will see that the Fleet Server is now connected. When you install Fleet Server on a host, it automatically installs the Elastic Agent as well to monitor system activity. This is perfect for us because we are installing everything on a single machine. If you were working in a production environment and wanted to install agents on multiple hosts, you'd continue enrolling Agents here. With the Fleet Server connected, you can now:

1. Exit the flyout menu
2. Refresh the page to see your newly installed Fleet Server and Elastic Agent
3. Click **Fleet Server Policy**



Agent Policies and Integrations: Agent policies allow you to define what data an Elastic Agent collects and how it behaves. Each policy is applied to enrolled agents and ensures consistent data collection across hosts. Policies are composed of integrations, which define what data is collected and how it is processed. You can see below that our agent currently has two integrations, Fleet Server and System. Both of these are native integrations for the Elastic Agent when you install Fleet Server, as we did in the previous step.

Fleet Server Policy		Revision	Integrations	Agents	Last updated on	Actions
Fleet Server policy generated by Kibana		1	2	1 agent	Feb 07, 2026	
Integrations Settings						
Search... Namespace Add Integration						
Integration policy ↑	Integration ↓	Namespace	Output	Actions		
fleet_server-1	Fleet Server v1.6.0	default	default	...		
system-1	System v2.12.0	default	default	...		

Confirming Log Ingestion

1. From the Kibana menu in the top-left corner, select **Discover**
2. Select the **logs-*** Data view in the top left to view incoming logs from your VM
3. From here, we can investigate interesting fields associated with our logs
4. View our incoming Linux logs

Answer the questions below

What is the name of the default integration used to collect system metrics and log data from your host?

System

In your VM terminal, create a new user with `useradd testuser`.

In Discover, enter the query `process.name: "useradd"` to highlight the `useradd` events. What is the `event.dataset` field value of the associated logs?

system.auth

Add `testuser` to the `sudoers` group with `gpasswd -a testuser sudo`. Enter the query `process.name: "gpasswd"` to highlight the log. What is the full message field value for the event created?

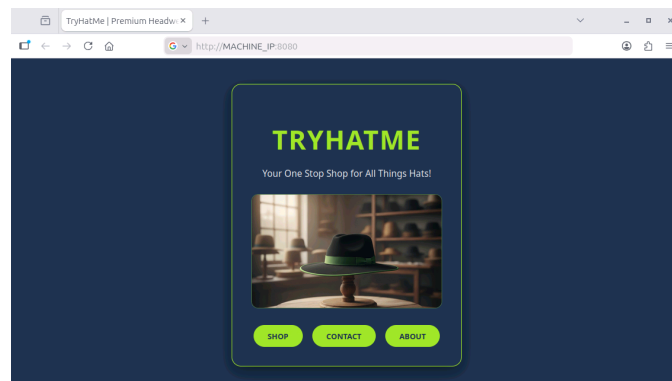
user testuser added by root to group sudo

Task 5 - Integrating Apache Web Logs

The TryHatMe Infrastructure: Imagine you've been hired to monitor a small but growing hat company named **TryHatMe**. They host their boutique shop on a Linux server, and their primary storefront is accessible on port `8080`. Before we dive into the logs, let's see what we are protecting. Navigate to the site now:

`http://MACHINE_IP:8080`

Take a moment to click around the shop, view the `About` page, and perhaps add a few items to your cart. Every click you make is recorded as an event by the server.



Every time a browser requests a page, the web server records the interaction in an access log file. Access logs list all sorts of useful information, such as the time of the request, the page you requested, and your IP address. Let's view the raw

logs in the terminal to see what the server sees. Go ahead and check out the following file:

- `/var/log/apache2/access.log`

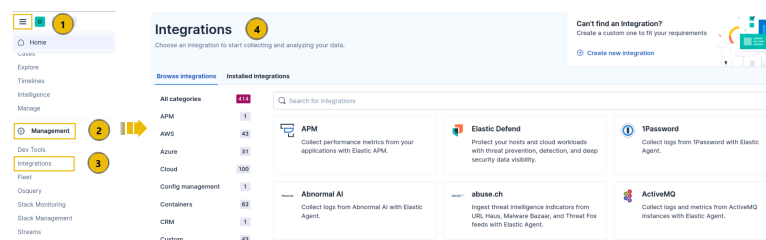
Investigating Access Logs

```
root@soclabelastic:/home/ubuntu/Downloads/elastic# tail /var/log/apache2/access.log
10.10.10.10 - - [30/Jan/2026:06:36:20 +0000] "GET /contact.html HTTP/1.1" 200 1021 "http://MACHINE_IP:8080/index.html" "Mozilla/5.0"
10.10.10.10 - - [30/Jan/2026:06:36:21 +0000] "GET /style.css HTTP/1.1" 200 991 "http://MACHINE_IP:8080/contact.html" "Mozilla/5.0"
. . .
```

Perhaps this log file isn't so tough to look at now since there are only a few logs, but imagine there are hundreds or thousands. By sending our logs to Elastic, we can make event analysis much easier.

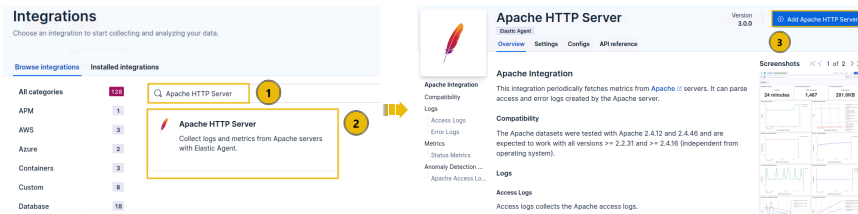
Elastic Integrations: Let's now explore Elastic integrations in more detail and begin ingesting Apache web logs:

1. Click the Kibana menu in the top left
2. Scroll down to **Management**
3. Select **Integrations**
4. Access the **Integrations** home page



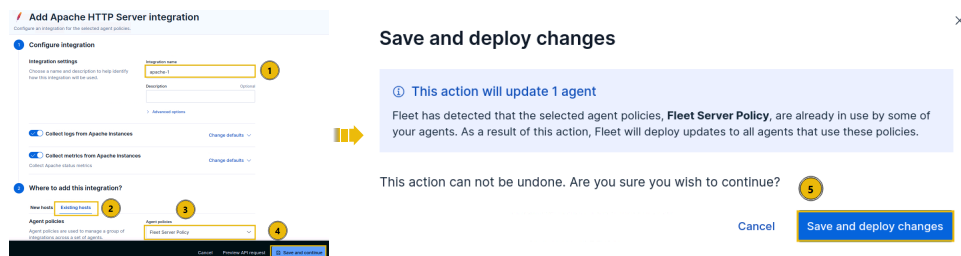
Apache HTTP Server Integration

1. Search **Apache HTTP Server**
2. Select the integration
3. Click **Add Apache HTTP Server**



Next, we will ensure that the integration is being applied to the correct Agent policy:

1. Keep the default name
2. Select **Existing hosts**
3. Ensure the **Fleet Server Policy** is selected
4. Click **Save and Continue**
5. Click **Save and deploy changes**



Confirming Log Ingestion

Let's head back to Discover and see how our web logs are coming in:

1. Select the **logs-*** Data view
2. Enter the query **event.module: "apache"**



Answer the questions below

What is the event.dataset field value for the Apache access logs?

apache.access

Head to /secret.html on the TryHatMe site. Using Discover, investigate the user_agent.original field from the recent requests. What is the hidden flag value?

THM{access_log_secrets!}

Task 6 - Managing Custom Log Types

Custom Log Types

2026-02-07T02:20:03Z	connection_start	a.dubois	72.14.36.2	10.10.10.115	uk-london-1
2026-02-07T02:22:33Z	connection_stop	s.novak	72.14.36.1	10.10.10.116	us-west-1
Date and Time	Connection Type	Username	IP Address	VPN Client IP	VPN Server
1	2	3	4	5	6

From the example above, we can identify six key fields that need to be parsed and managed in Elasticsearch. While our next integration makes it straightforward to monitor and ship custom log files, we still need to ensure these fields are properly extracted so they can be searched and analyzed in Kibana.

In a real environment, these logs would be generated in real time as users authenticate to a VPN service. For this lab, we'll simulate the activity by running a

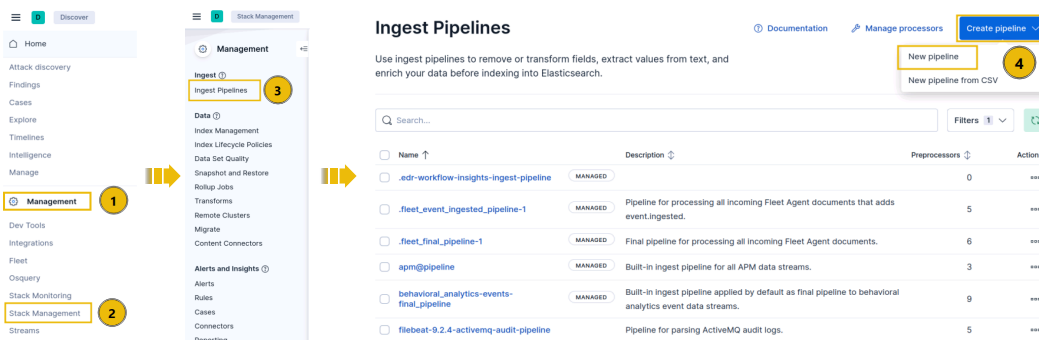
script that generates sample log data. The script will create a file containing 500 VPN log entries at `/var/log/vpnlog`. Make sure you're using the `root` user:

1. Run the Python script to generate the VPN logs `python3 /home/ubuntu/Downloads/scripts/vpnlog.py`
2. Check out the logs in your terminal `tail /var/log/vpnlog`

Building an Ingest Pipeline: Before we instruct Elastic where to ingest our logs from, we need to tell it how to handle the custom logs. Let's head back to Kibana to create an ingest pipeline. Ingest pipelines allow us to process incoming events before they are indexed, enabling us to extract structured fields from raw log data. In our previous System and Apache integrations, this parsing was handled automatically by Elastic. However, because we're working with a custom log source, we need to explicitly define how Elasticsearch should parse and manage this data.

From the Kibana menu:

1. Scroll down to `Management`
2. Select `Stack Management`
3. Select `Ingest Pipelines`
4. Click `Create pipeline` → `New pipeline`



In the Create pipeline flyout menu:

1. Name your pipeline `vpn.logs.pipeline`
2. Give a short description
3. Click `Add a processor`

Create pipeline

[Create pipeline docs](#)

Name

A unique identifier for this pipeline.

Name

vpn.logs.pipeline

Description

A description of what this pipeline does.

Description (optional)

A custom pipeline for VPN Logs

Processors

Add your first processor

Use processors to transform data before indexing. [Learn more.](#)

Add a processor

> Add version number

Enabled

> Add metadata

Enabled

How to use this pipeline during data ingestion

Adding Processors: Processors define the actions Elasticsearch takes on an event as it is ingested. The Grok processor extracts multiple fields from the raw `message` field by matching specific parts of each log entry. The Date processor converts our `event.time_string` field into the native `@timestamp` field. Now add these processors:

1. **GrokField:** `message` Pattern: `%{TIMESTAMP_ISO8601:event.time_string} %{WORD:event.action} %{USER:user.name} %{IP:source.ip} %{IP:vpn.client.ip} %{NOTSPACE:vpn.server.region}`
2. **DateField:** `event.time_string` Formats: `ISO8601` Target field: `@timestamp`

Add processor

Grok documentation

Processor

Grok

Field

message

Patterns

`%{TIMESTAMP_ISO8601:event.time_string} %{WORD:event.action} %{USER:user.name} ;`

Pattern definitions (optional)

Cancel

Add processor

Add processor

Date documentation

Processor

Date

Field

event.time_string

Formats

ISO8601

Target field (optional)

@timestamp

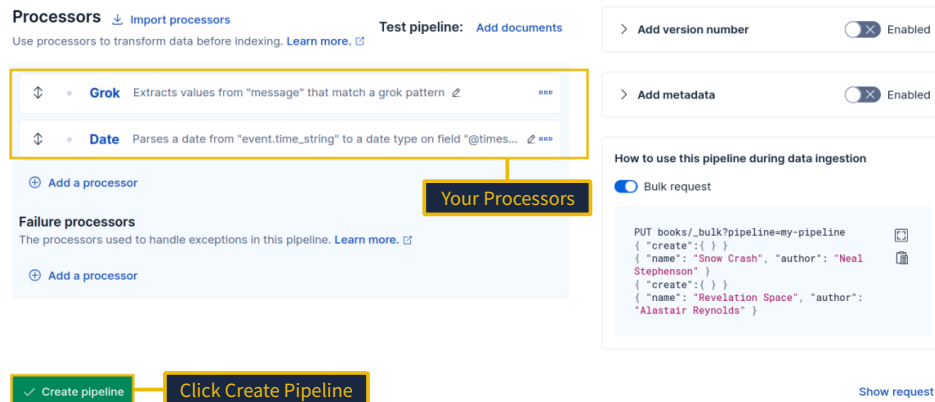
Timezone (optional)

Cancel

Add processor

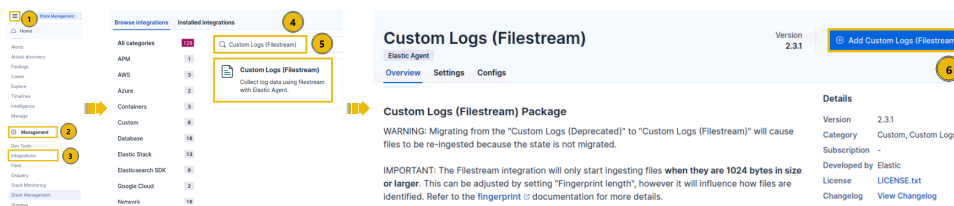
After adding both processors, we should see them in the Create pipeline home screen:

- Click `Create pipeline`



Filestream Integration: Now that we have the ingest pipeline built, we can work to get our VPN logs into Elastic using another integration. Let's head back to the integration menu and take a look:

1. Click the Kibana menu in the top left
2. Scroll down to **Management**
3. Select **Integrations**
4. Search **Custom Logs (Filestream)**
5. Select the integration
6. Click **Add Custom Logs (Filestream)**



Next, we will apply the integration to the correct Agent policy, use our ingest pipeline, and ensure it points to our newly created **vpnlog** file:

1. Keep the default name
2. Select the **Change defaults** dropdown
3. Enter the **/var/log/vpnlog** path
4. Enter **vpn.logs.pipeline** to match the ingest pipeline we created

5. Select **Existing hosts**
6. Ensure the **Fleet Server Policy** is selected
7. Click **Save and Continue**
8. Click **Save and deploy changes**

Add Custom Logs (Filestream) integration

Configure an integration for the selected agent policies.

1 Configure integration

Integration settings
Choose a name and description to help identify how this integration will be used.

Integration name **1**
filestream-1

Description Optional

[Advanced options](#)

☒ **Custom Filestream Logs** **2**
Custom Filestream Logs [Change defaults ^](#)

Paths **3**
/var/log/vpnlog

[Add row](#)

4 Ingest Pipeline Optional
vpn.logs.pipeline
The Ingest Node pipeline ID to be used by the integration. This field is ignored when 'Use the "logs" data stream' is enabled.

2 Where to add this integration?

New hosts **5**
Existing hosts **6**

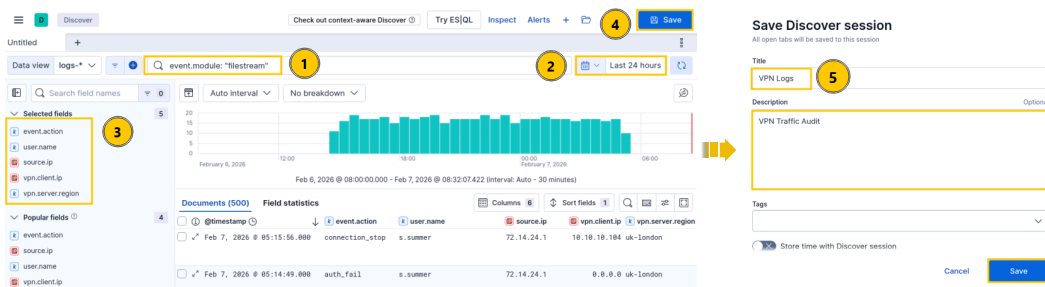
Agent policies
Agent policies are used to manage a group of integrations across a set of agents.

Agent policies **7**
Fleet Server Policy
1 agent is enrolled with the selected agent policies.

[Cancel](#) [Preview API request](#) [Save and continue](#)

Confirming Log Ingestion and Parsing: To confirm that we have correctly configured the integration and our ingest pipeline, let's pivot back to Discover:

1. Enter the query `event.module: "filestream"`
2. Set the time to `Last 24 hours`
3. Build a table with our new fields by clicking the `+` next to the following fields `event.actionuser.namesource.ipvpn.client.ipvpn.server.region`
4. Click `Save`
5. Give your Discover session the name `VPN Logs` and a description, then save, as we will use it in the next task



Answer the questions below

Investigate the newly ingested VPN log data. Who is the most active user on the network?

s.summer

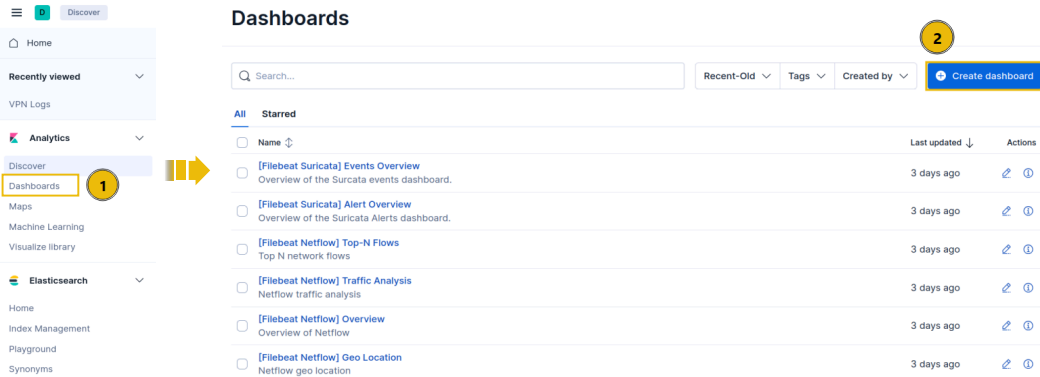
What is the source.ip of the user you identified in the previous question?

72.14.24.1

Task 7 - Creating Dashboards and Visualizations

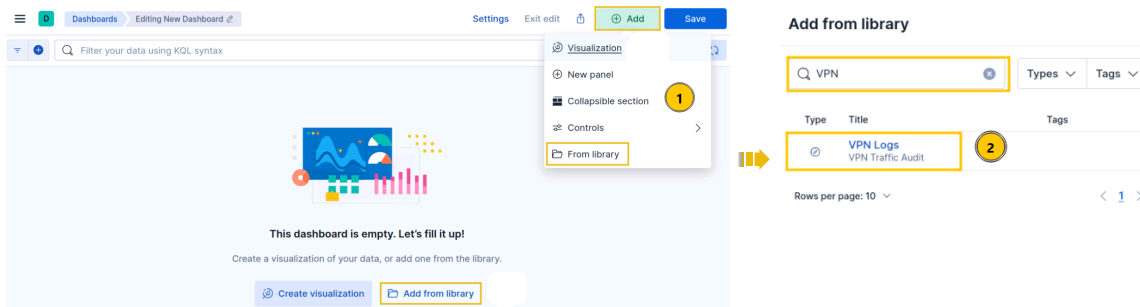
Creating the Dashboard: From Discover, go ahead and click the Kibana sidebar menu:

1. Select `Dashboards`
2. Click `Create dashboard`



Our new dashboard is looking pretty barren at the moment, so let's go ahead and add some data by adding the search we saved in the previous task:

1. Select **Add from library** in the top right or at the bottom of the page
2. Search for and select the saved search, **VPN Logs**, from the previous task



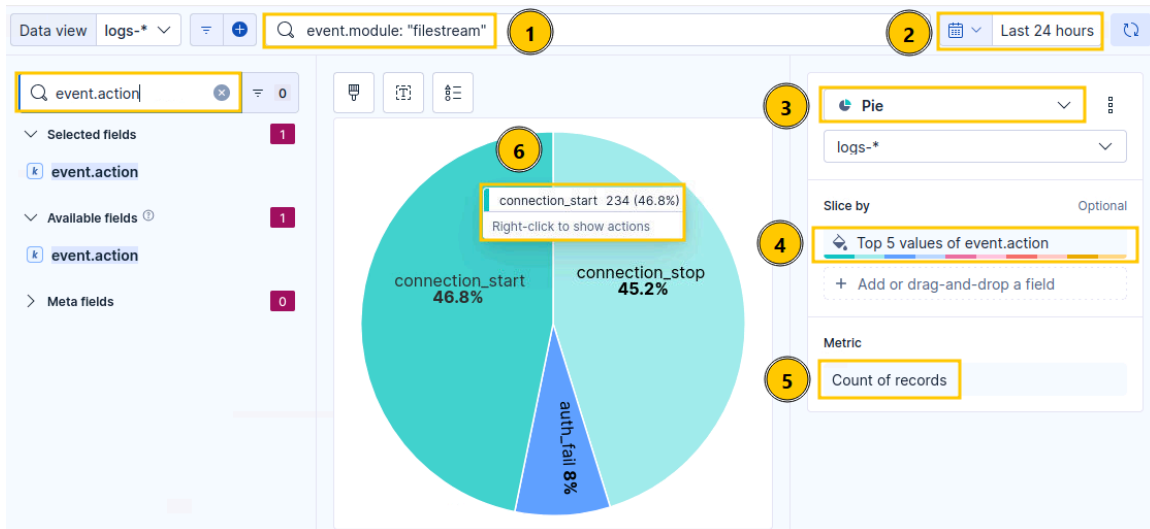
We now have a clean overview of the VPN log data that was ingested in the previous task.

VPN Logs ⓘ					
500 documents					
ⓘ @timestamp ⌚	⌵ event.action	ⓘ user.name	ⓘ source.ip	ⓘ vpn.client.ip	ⓘ vpn.server.region
Feb 7, 2026 @ 05:15:56.000	connection_stop	s.summer	72.14.24.1	10.10.10.104	uk-london
Feb 7, 2026 @ 05:14:49.000	auth_fail	s.summer	72.14.24.1	0.0.0.0	uk-london
Feb 7, 2026 @ 05:13:44.000	connection_start	f.ahmed	72.14.37.1	10.10.10.117	uk-london
Feb 7, 2026 @ 05:12:06.000	connection_stop	c.11n	72.14.23.1	10.10.10.103	us-west-1

Adding Visualizations: While the above view is useful, it closely resembles what we could already see in Discover. To gain deeper insight and highlight patterns

more effectively, we'll add visualizations to enhance our view of the log data. On the top right, click **Add** then **Visualization**. Let's focus on the **event.action** field, which tracks connections to our VPN service:

1. Enter the query **event.module: "filestream"**
2. Set the time to **Last 24 hours**
3. Change the chart type to **Pie**
4. Drag or select the **event.action** field for **Slice by**
5. Set the Metric to **Count**
6. Check out the **connection_start** field value, which accounted for 46.8% or 234 records

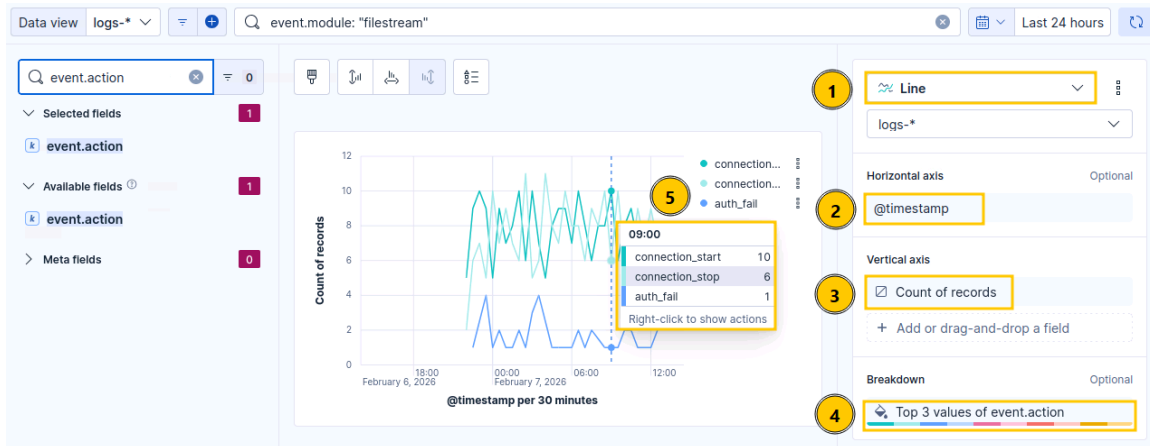


Enhanced Visualizations: This is very handy for giving us a quick look at the total number of connections over our selected time period, but what if we want a more detailed look? Perhaps we want to see the connections as they occurred over time. Let's make some changes to our current visualization:

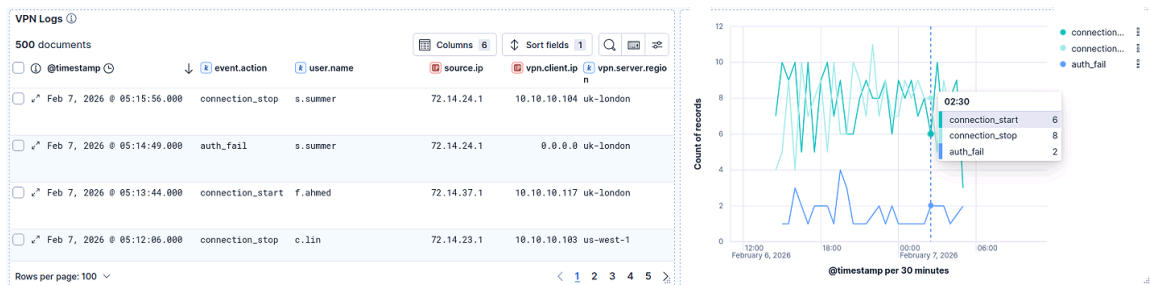
1. Change the graph type to **Line**
2. **Horizontal axis** to **@timestamp**
3. Keep the **Vertical axis** the same
4. Set the **Breakdown** to **event.action**

5. Now we can view the total for each `event.action` for a given 30-minute time period

Go ahead and click `Save and return` in the top right corner.



Kibana dashboards are fully modular, allowing analysts to drag and drop searches and visualizations into layouts that best support investigation and monitoring. Let's practice creating a few more by answering the questions below.



Answer the questions below

Create a new Pie chart visualization using the search `event.action: "auth_fail"`. In Slice by, add the `user.name` field and set the Metric to Count. What percent of `auth_fail` events does the user `p.mallow` account for? (e.g., 82.7%)

62.5%

Using your dashboard, experiment with creating a visualization for the `vpn.server.region` field. Which field value represents the least accessed `vpn.server.region`?

us-west-1

Investigate the `vpn.client.ip` field. Which `vpn.client.ip` was only logged 26 times within the available data?

10.10.10.116

Task 8 - Conclusion

- **Installation:** Setting up Elasticsearch (the search and analytics engine) and Kibana (the visualization interface) on a **Linux** host.
- **Data Ingestion:** Collecting native Linux system logs and **Apache** web logs using **Fleet Server** and an **Elastic Agent**.
- **Advanced Integration:** Onboarding a custom **VPN** log source to expand data coverage.
- **Visualization:** Creating dashboards and visualizations in Kibana to transform raw log data into actionable security insights.